Tokenization và text classification

Từ nhận xét lớp học đến một baseline đọc được lỗi

Chương 6 · Bài 6.1

Từ nhận xét đến nhãn

Nhận xét: “ví dụ hay”, “bài giảng khó hiểu”, “không tệ”.

Có vài nghìn nhận xét. Muốn tìm những phần cần sửa trước.

Chuỗi ký tự → token → vector → nhãn.

Câu hỏi: bước nào làm mất thông tin khi biến câu thành số?

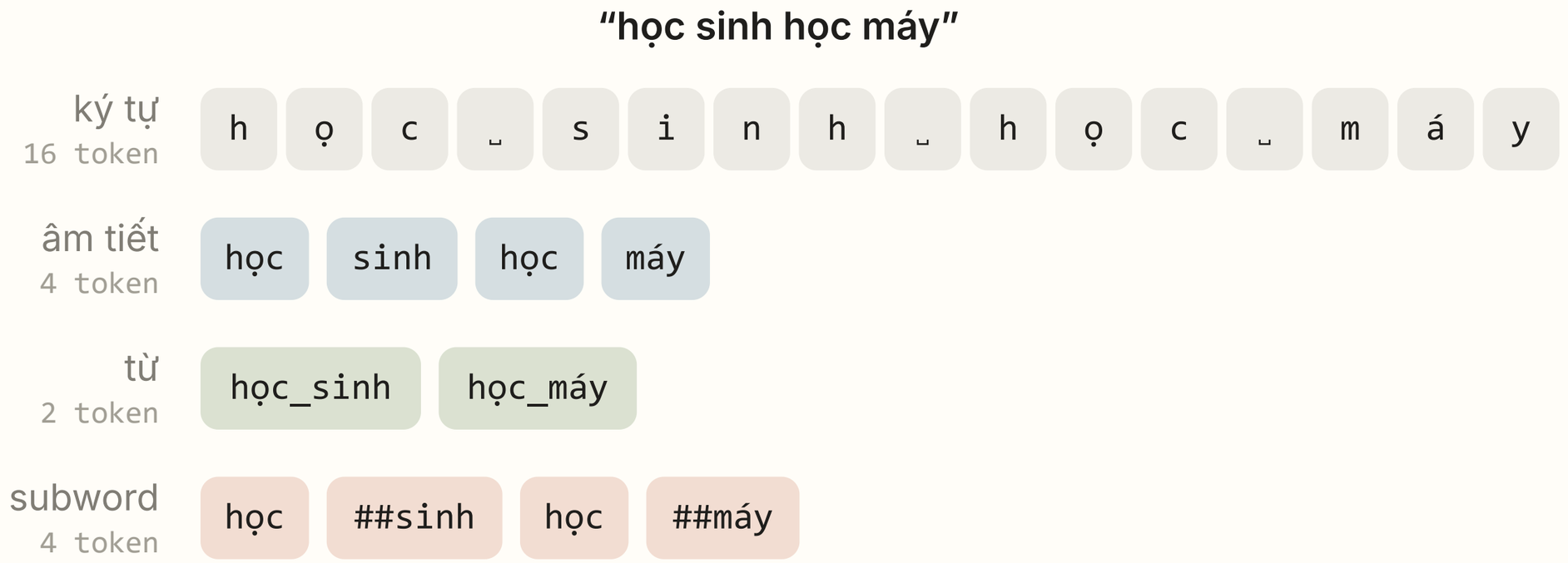
Mục tiêu buổi này

- Kiểm cách cắt tiếng Việt, chuẩn hóa NFC.
- Làm một bước BPE; mã hóa bằng WordPiece.
- Dựng TF-IDF cùng logistic regression đúng split; so với Naive Bayes.
- Đọc lỗi phủ định và từ lạ.

01

Câu trở thành token

Khoảng trắng chưa chắc là ranh giới từ



cùng một câu, số token tùy đơn vị tokenizer chọn

Câu “học sinh học máy” ở bốn mức: 16 ký tự, 4 âm tiết, 2 từ (sau tách từ), hoặc các mảnh subword; cùng một câu, số token tùy tokenizer.

Unicode và token IDs

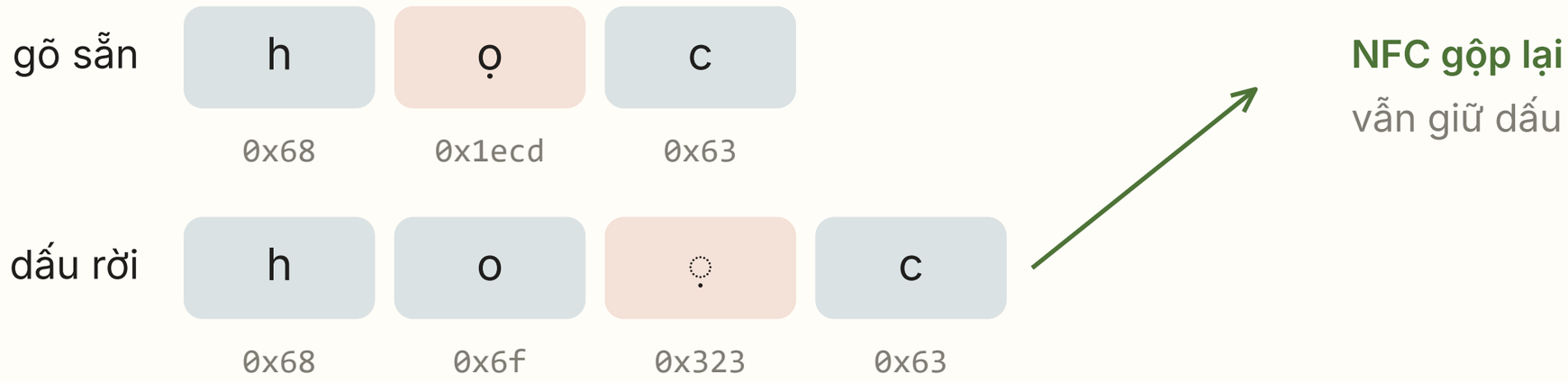
```
print(normalize_text(sentence).split())
nfc = unicodedata.normalize("NFC", "ho\u0323c")
print(nfc == "học")
```

KẾT QUẢ

```
['học', 'sinh', 'học', 'máy']
True
```

- NFC gộp “o” và dấu nặng rời thành “ọ”, vẫn giữ dấu.
- ID là địa chỉ trong bảng, không phải độ lớn ý nghĩa.
- Tokenizer đi cùng model: PhoBERT cần `học_sinh`.

Hai cách ghi chữ “ọ”



Hai chuỗi cùng hiện là “học”: bản gõ sẵn có 3 ký tự, bản dấu rời có 4 (chữ o rồi dấu nặng U+0323); NFC gộp bản thứ hai về bản thứ nhất mà vẫn giữ dấu.

HỎI NHANH

`"học sinh".split()` trả về hai mảnh. Từ đó ta rút ra được điều gì?

A Trong tiếng Việt, giữa hai khoảng trắng luôn là đúng một từ

B Đây là hai âm tiết, còn theo ngữ cảnh có thể chúng là một từ

C Phải bỏ dấu trước, rồi mới ghép được thành một từ

D Model nào cũng cần nối các âm tiết bằng dấu _

HỎI NHANH

Chuỗi `b` in ra trông như “học”, nhưng `b == "học"` lại là `False`, và `[hex(ord(ch)) for ch in b]` cho `['0x68', '0x6f', '0x323', '0x63']`. Trước khi đếm token, nên xử lý thế nào?

A Dấu nặng đang tách rời khỏi chữ “o”, nên chuẩn hóa NFC

B Ký tự `0x323` là rác, nên xóa mọi ký tự không phải chữ cái

C Chuỗi bị lỗi mã hóa, nên bỏ dấu cả hai chuỗi rồi so

D Hai chuỗi khác chữ hoa thường, nên gọi `lower()` rồi so lại

02

Subword cho từ hiếm

Một bước BPE

Train: `low` 3 lần, `lower` 2 lần, `new` 1 lần.

Cặp `l,o`: lần.

Phá hòa theo thứ tự từ điển: gộp `l,o` trước. Lặp đến khi đủ từ vựng.

KẾT QUẢ

```
('l', 'o') 5
low ['lo', 'w']
lower ['lo', 'w', 'e', 'r']
new ['n', 'e', 'w']
```

Đếm có trọng số, không đếm mỗi loại từ một lần.

HỎI NHANH

Sau bước gộp `l,o`, corpus gồm `lo w` 3 lần, `lo w e r` 2 lần, và `n e w` 1 lần. Bước BPE thứ hai gộp cặp nào?

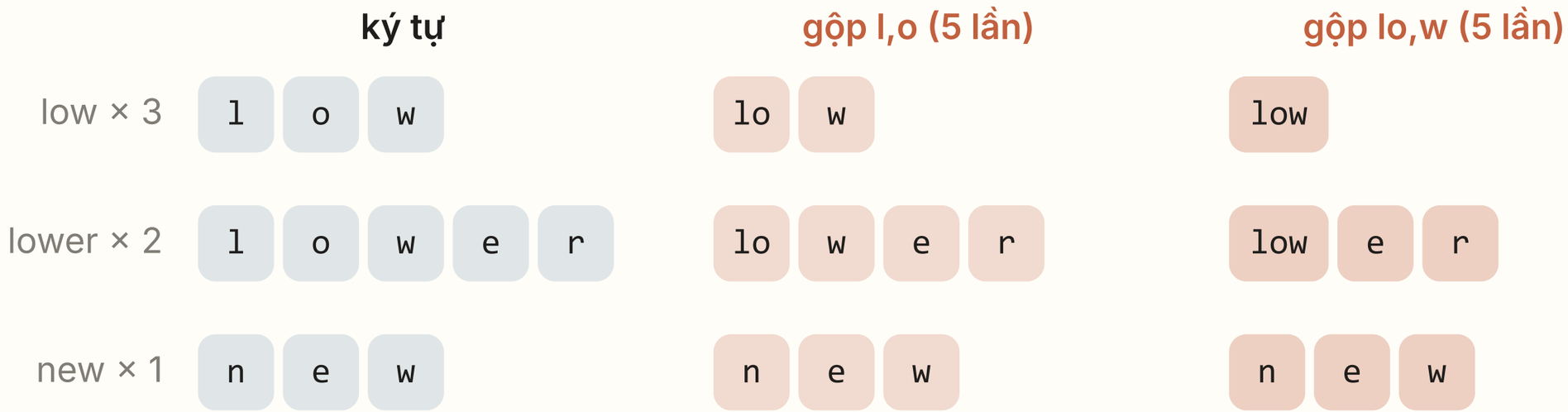
A `lo,w`, xuất hiện 5 lần

B `o,w`, xuất hiện 5 lần

C `w,e`, xuất hiện 2 lần

D `e,r`, xuất hiện 2 lần

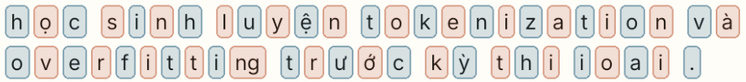
Hai bước gộp



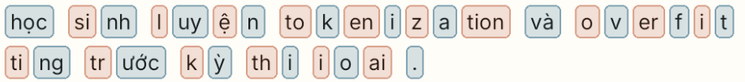
tần suất có trọng số: l,o có trong low (3) và lower (2), cộng 5

BPE trên corpus của bài: low ×3, lower ×2, new ×1. Bước một gộp l,o (5 lần), bước hai gộp lo,w (5 lần), nên low thành một token.

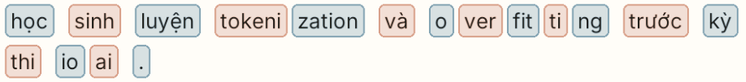
BPE học trên lời giảng của khóa



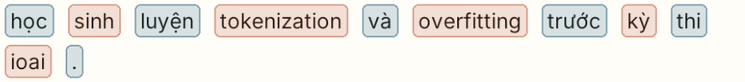
từ vựng 149 mảnh (chỉ có ký tự): câu thành 51 token



từ vựng 300 mảnh: câu thành 33 token



từ vựng 1200 mảnh: câu thành 17 token



từ vựng 4000 mảnh: câu thành 11 token

BPE học trên chính lời giảng của giáo trình (khoảng 327 nghìn từ): từ vựng càng lớn, câu càng ít token; từ hiếm như “tokenization” vẫn ghép được từ mảnh quen.

WordPiece: mảnh dài nhất, không quay lui

Từ vựng có `play`, `##ing`: `playing` thành `play`, `##ing`.

`##` đánh dấu mảnh nối tiếp (BERT).

Khi học: chọn cặp có ——— cao nhất.

Không mảnh nào khớp: cả từ thành `[UNK]`.

BPE lưu luật gộp; WordPiece chỉ giữ vocabulary cuối.

GPT-2: BPE trên byte, không cần token unknown.

HỎI NHANH

Từ vựng WordPiece chỉ có `un`, `una`, `##able` và `[UNK]`. Mã hóa từ `unable` thì được gì?

A `un, ##able`

B `una, ##ble`

C `una, ##b, ##le`

D `[UNK]`

Tham lam rồi kẹt

từ vựng `un` `una` `##able` `[UNK]`

unable `una` + ble ? ##ble, ##bl, ##b đều không có

→ `[UNK]` tham lam lấy mảnh dài nhất, không quay lui thử un + ##able

WordPiece mã hóa tham lam: lấy “una”, mảnh dài nhất khớp đầu từ; phần “ble” không khớp mảnh nào nên cả từ thành token UNK, dù cách cắt un + ##able có tồn tại.

03

Một baseline từ vựng

TF-IDF

Số lần xuất hiện nhân IDF, rồi chuẩn hóa L2 từng câu.

Mục phổ biến giảm trọng số tương đối. Công thức không tự hiểu nghĩa: hai từ đồng nghĩa vẫn ở hai cột.

HỎI NHANH

Sau TF-IDF có chuẩn hóa L2 mặc định, nhận xét “hay” và nhận xét “hay hay hay” cho ra hai vector thế nào?

A Vector thứ hai gấp ba lần vector thứ nhất

B Hai vector bằng nhau

C Vector thứ hai gấp $\sqrt{}$ lần vector thứ nhất

D Hai vector khác nhau, vì IDF tính theo số lần lặp trong câu

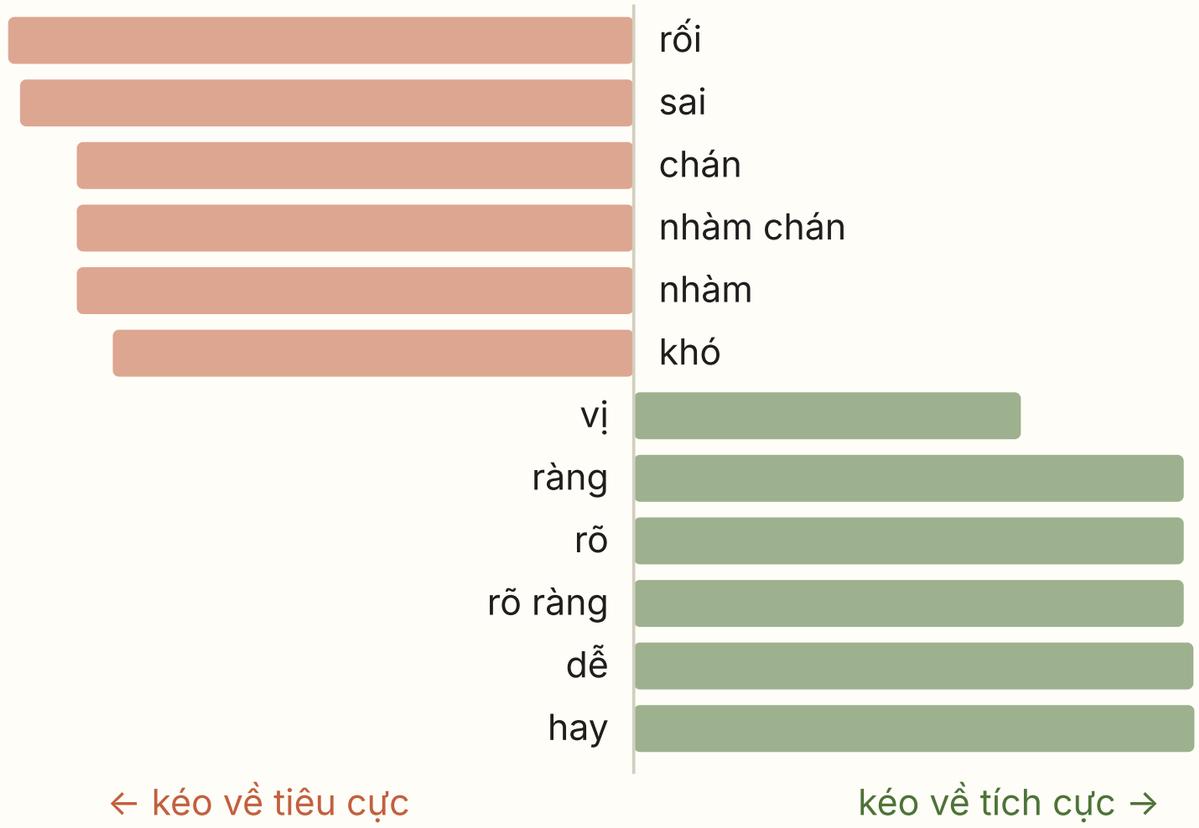
Logistic regression đọc vector

Nhãn 1: tích cực.

Nhãn 0: tiêu cực.

Unigram bỏ thứ tự; bigram giữ một phần thứ tự.

Model đã học từ nào



Sáu đặc trưng có trọng số âm nhất và sáu dương nhất của logistic regression trên TF-IDF (học từ 12 câu train của bài): toàn từ khen, chê trực tiếp; không có cột nào cho “không”.

Naive Bayes: học bằng đếm

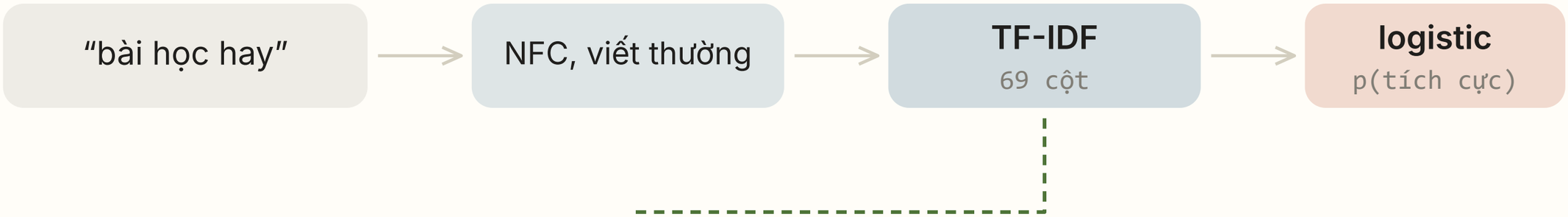
Làm trơn Laplace : không có xác suất 0.

Giả định: token độc lập khi biết nhãn.

“theo” gần như luôn đi với “dõi”: bằng chứng bị cộng hai lần, xác suất quá tự tin.

Vẫn là bộ phân loại tuyến tính trên vector đếm.

Train học, validation biến đổi



validation chỉ transform; từ chưa gặp như “không” bị bỏ

Pipeline baseline: chuẩn hóa văn bản, TF-IDF với vocabulary và IDF fit trên train (69 cột), rồi logistic regression cho xác suất tích cực.

Dùng Pipeline

```
baseline = Pipeline([
    ("tfidf", TfidfVectorizer(
        preprocessor=normalize_text,
        ngram_range=(1, 2))),
    ("classifier", LogisticRegression(
        C=4.0, random_state=0, max_iter=1000)),
])
baseline.fit(train_texts, train_labels)
```

12 câu train, 6 câu validation tự soạn.

Mỗi câu validation đổi accuracy

.

Đủ để soi lỗi, quá ít để so hai model.

Kết quả thật

KẾT QUẢ

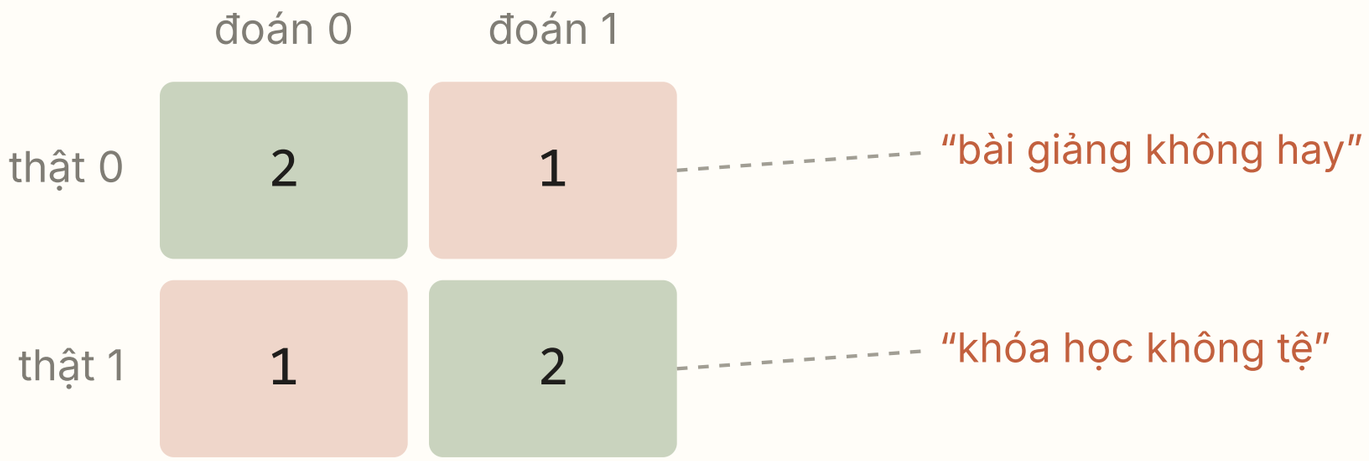
```
shape train: (12, 69)
accuracy: 0.667
macro-F1: 0.667
[[2 1]
 [1 2]]
```

69 cột: 31 âm tiết và 38 bigram của train.

Hàng: nhãn thật. Cột: nhãn đoán.

4/6 câu đúng, mỗi lớp có một câu bị nhầm.

Hai ô lệch là hai câu nào



accuracy 4/6 = 0.667, macro-F1 0.667

Confusion matrix của baseline trên sáu câu validation (số từ code bài): hai ô lệch là hai câu chứa “không”, mỗi câu đoán ngược một chiều.

HỎI NHANH

Ta muốn chạy 5-fold cross-validation cho baseline trên cả 18 câu có nhãn. Làm cách nào thì điểm mới tin được?

A Fit `TfidfVectorizer` trên cả 18 câu, rồi CV logistic trên ma trận

B Fit `TfidfVectorizer` riêng trên phần validation của từng fold

C Giữ vectorizer đã fit trên 12 câu train, chỉ CV phần logistic

D Đưa cả `Pipeline` cùng văn bản thô vào `cross_val_score`

Từ phủ định đã bị bỏ

KẾT QUẢ

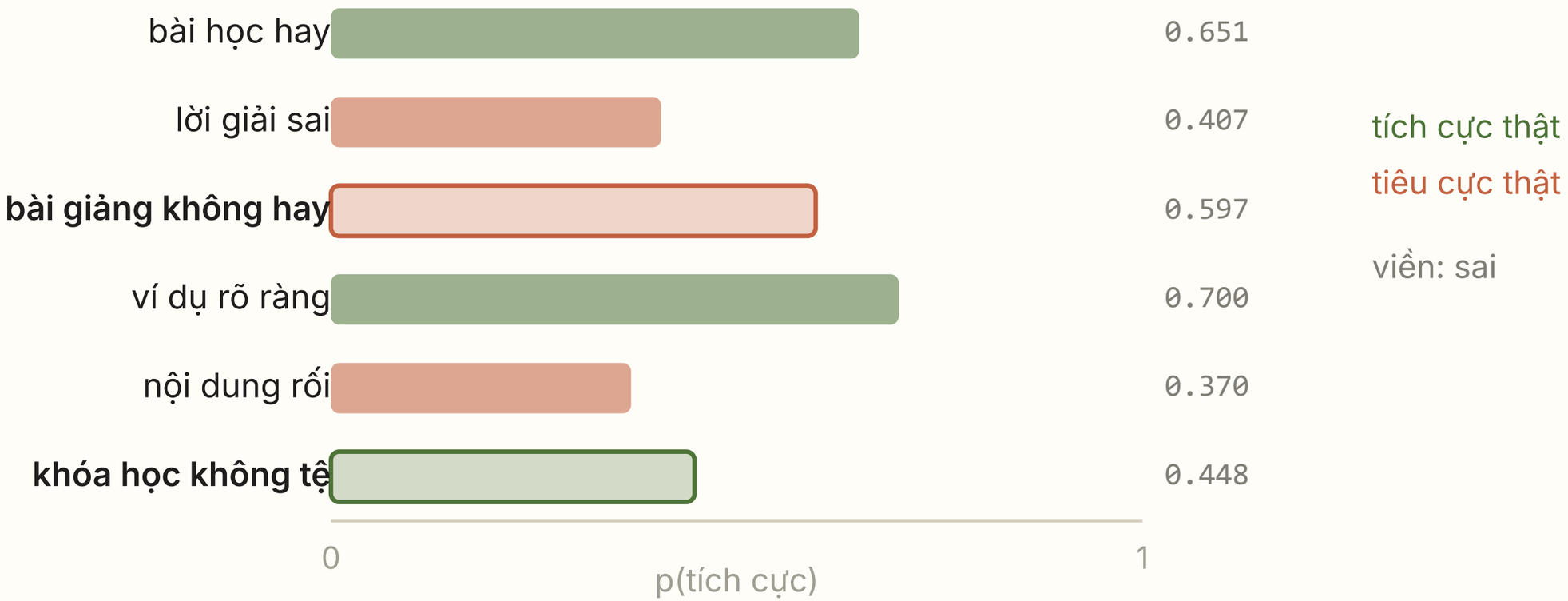
```
thật=0 đoán=1 p1=0.597 | bài giảng không hay
thật=1 đoán=0 p1=0.448 | khóa học không tệ
'không' có trong vocabulary: False
```

Từ lạ không có cột.

Bigram chưa học cũng không có cột.

Đọc token và feature trước khi đổi model.

Xác suất trên sáu câu



Xác suất tích cực baseline gán cho sáu câu validation (số từ code bài): hai câu sai đều chứa “không”, từ chưa có trong vocabulary, nên model chỉ còn thấy “hay” hoặc “tệ”.

Đổi classifier không cứu được

```
naive_bayes = Pipeline([
    ("count", CountVectorizer(
        preprocessor=normalize_text,
        ngram_range=(1, 2))),
    ("classifier", MultinomialNB(alpha=1.0)),
])
```

KẾT QUẢ

```
NB đoán: [1 0 1 1 0 0]
thật:    [1 0 0 1 0 1]
```

Sai đúng hai câu “không”.

Cùng vocabulary nên cùng lỗi.

HỎI NHANH

Cả logistic regression lẫn Naive Bayes đều đoán ngược hai câu có chữ “không”. Bước tiếp theo nào nhắm đúng nguyên nhân?

A Thử thêm một classifier mạnh hơn, như gradient boosting

B Hạ threshold để các câu phủ định rơi về lớp 0

C In ra các feature còn lại của câu sau `transform`

D Tăng `C` để model chú ý hơn tới các từ hiếm

Tính tay một TF-IDF

3 câu: “hay dễ”, “hay rõ”, “khó”.

IDF của “hay” là 1.288, của “dễ” là 1.693.

Sau L2: .

Trọng số **giả định** , bias 0:

, .

Nếu “hay” có ở cả ba câu: IDF bằng 1, giảm tương đối chứ không bị xóa.

HỎI NHANH

Giữ IDF như trên, trọng số giả định là và bias bằng 0. Câu “hay hay dễ” cho xấp xỉ bao nhiêu?

A 0.452, giống câu “hay dễ”

B 0.707

C 0.571

D 0.500, vì chuẩn hóa xóa hiệu ứng lặp

Khi đi thi

- TF-IDF cùng logistic regression trước: mốc và file nộp hợp lệ.
- Chia validation giống cách tạo test; loại bản trùng.
- Đổi một thành phần mỗi lần, cùng split, đúng metric.
- Đọc câu sai, feature, số câu thành vector 0.
- Đếm độ dài bằng đúng tokenizer của model.
- Giữ thứ tự ID test và bảng ánh xạ nhãn.

04

Luyện đề

LUYỆN ĐỀ 1

Câu được mã hóa thành `[BOS] + từ + [EOS]`, dài quá thì cắt `tokens[:max_len]`. Với `max_len = 8` và một câu 9 từ, lỗi chính là gì?

A Mất EOS, chỉ còn BOS và 7 từ

B PAD bằng 0 nên loss tính cả vị trí đệm

C Từ vựng theo từ quá lớn, phải dùng BPE

D Câu ngắn bị pad thiếu một vị trí

PHỎNG THEO NOAI 2026 Singapore · Chung kết · Câu 19

LUYỆN ĐỀ 2

Bình luận sản phẩm có nhiều tên hãng mới và lỗi gõ như “đẹppp”. Tokenizer theo từ đã có 300 nghìn mục mà test vẫn đầy từ lạ. Chuyển sang subword như BPE hay WordPiece thì được lợi chính ở đâu?

A Chạy nhanh hơn, vì chuỗi subword luôn ngắn hơn chuỗi từ

B Cài đặt đơn giản hơn, vì không phải học từ vựng trên train

C Không có lợi với tiếng Việt, vì subword chỉ học được trên tiếng Anh

D Từ lạ được ghép từ mảnh quen, từ vựng nhỏ, từ cùng gốc dùng chung mảnh

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 235

LUYỆN ĐỀ 3

Corpus là list từ `["bun"]*5 + ["sun", "sing", "king", "ring"]`. Một bạn đếm các cặp ký tự kề nhau trong vòng `for word in set(corpus)`, và phá hòa theo thứ tự từ điển. Bước gộp BPE đầu tiên đúng ra là gì, và code này gộp gì?

A Đúng ra là `i,n` với 3 lần, code gộp `u,n` với 2 lần

B Đúng ra là `u,n` với 6 lần, code gộp `i,n` với 3 lần

C Cả hai đều gộp `u,n`, chỉ khác số đếm

D Cả hai đều gộp `i,n`, chỉ khác số đếm

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Part 5.1–5.2

Chốt ba điều

Tokenization quyết định thông tin nào vào được model.

TF-IDF là biểu diễn từ vựng; logistic regression và Naive Bayes đọc cùng vector.

Hai model cùng sai một câu: xem biểu diễn trước khi đổi model.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook và đọc hai câu phủ định trước khi thử một thay đổi có kiểm chứng.